CYBEROPS ASSOCIATE INTERNSHIP · NETACAD

SOC Analyst Project

Privileged Access Enforcement & Threat Detection

Validating a PAM control under live enforcement and simulating a full attack kill-chain — both monitored, captured, and triaged end-to-end with Security Onion (Squert, Kibana).

JumpServer | Security Onion | Red Hat | Squert | Kibana | Kali Linux | Metasploitable2

Prepared by **Amr Ahmed Mohamed Abdeldayem**

EXECUTIVE SUMMARY

What this project set out to prove

Two independent, realistic scenarios were built on the same monitored network segment to demonstrate both sides of SOC work: enforcing a preventive control, and detecting an active compromise. Every packet on the segment was captured passively by Security Onion and triaged the way a working analyst would alert, payload, verdict.

2	115	1,861
attack scenarios built	NIDS alerts triaged	network events logged

01

Deploy & harden

Stand up a PAM bastion (JumpServer) with active command-filtering enforcement.

02

Build the range

Wire a 4-VM segment plus a passive sensor with full mutual reachability.

03

Simulate a kill chain

Run real recon and exploitation from Kali against Metasploitable2.

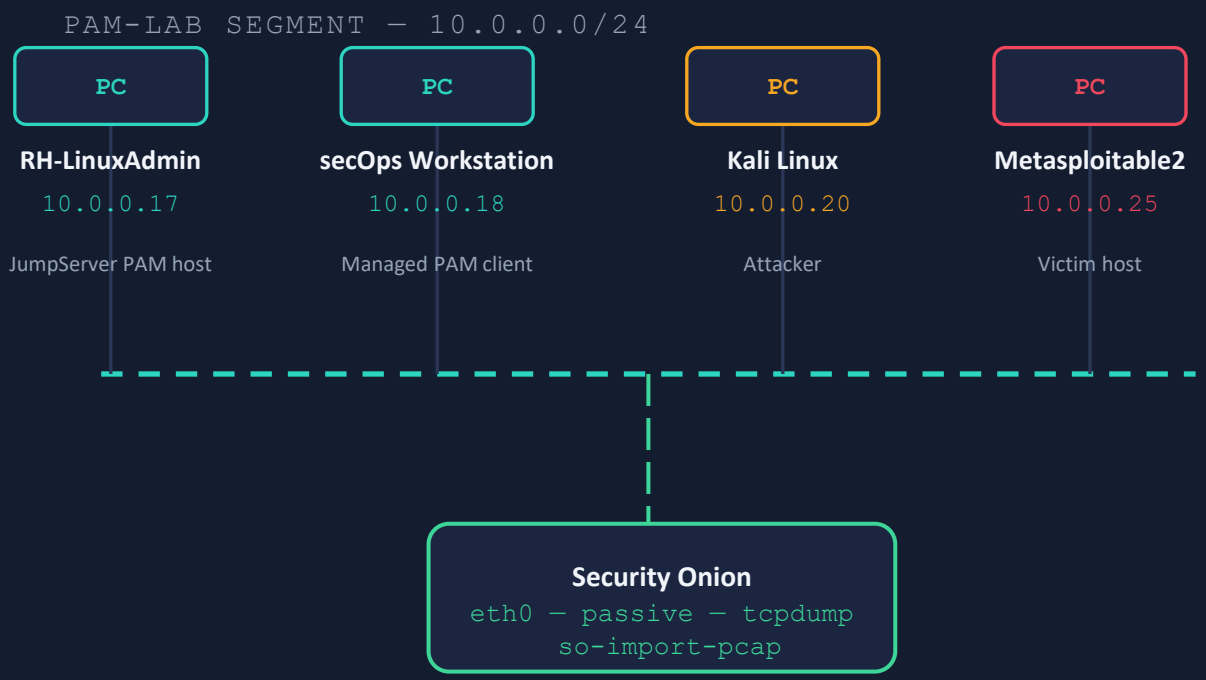
04

Triage like an analyst

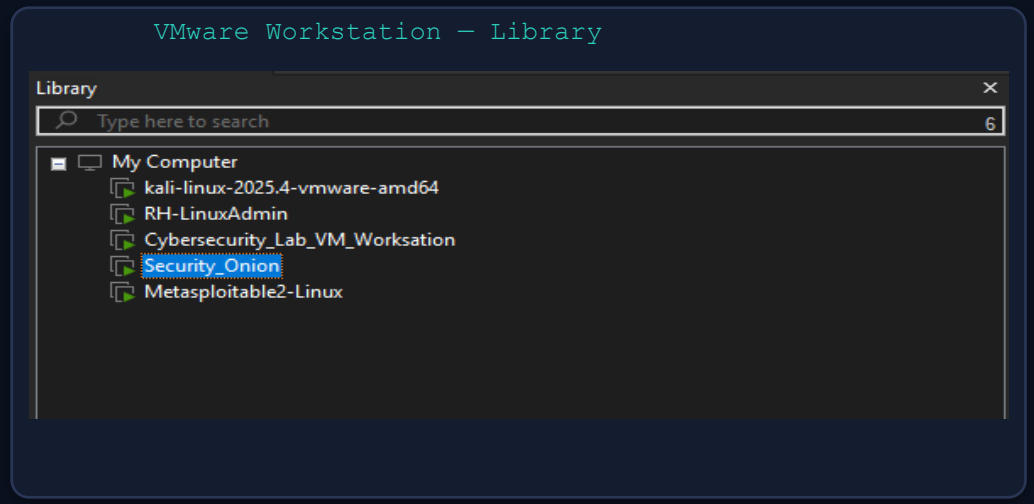
Pull every alert's payload and defend a true-positive / false-positive verdict.

ENVIRONMENT

Lab architecture & network topology



All four hosts share one broadcast segment; Security Onion listens passively on eth0 — every packet between every pair of hosts is recorded, regardless of which scenario generated it.



Five VMs total — the pam-lab segment uses four of them plus the Security Onion sensor shown here, confirming the exact host inventory used for both scenarios.

- **Security Onion** Squert + Kibana
- **RH-LinuxAdmin / secOps** PAM bastion + managed client (Scenario 1)
- **Kali / Metasploitable2** Attacker + vulnerable target (Scenario 2)

METHODOLOGY

One monitored segment, two test scenarios

SCENARIO 1 — BLUE TEAM

PAM Enforcement Validation

Prove that a privileged-access bastion actively blocks dangerous commands in real time, then confirm what network monitoring can and cannot see about that session.

1. Deploy JumpServer on RH-LinuxAdmin (.17)
2. Connect secOps client (.18) through it
3. Attempt 6 high-risk commands
4. Capture the full session with tcpdump
5. Import into Security Onion & triage alerts

SCENARIO 2 — RED VS. BLUE

Offensive Simulation & Detection

Run a real, minimal kill chain — reconnaissance through exploitation — against a deliberately vulnerable host, and validate that the sensor catches it.

1. Assign Metasploitable2 a static IP (.25)
2. Start capture before any attack traffic
3. Nmap service/version scan from Kali (.20)
4. Exploit the vsftpd 2.3.4 backdoor
5. Confirm root compromise in Squert

PART 1 — PAM ENFORCEMENT

JumpServer: bastion host & command filtering

A JumpServer instance was deployed on RH-LinuxAdmin (10.0.0.17) as a centralized Privileged Access Management bastion. The secOps workstation (10.0.0.18) was on-boarded as a managed client, so every privileged session to the host is proxied, recorded, and now filtered through JumpServer rather than reaching the target directly.

Command filter policy

A rule set of high-risk commands was attached to the managed asset's protocol settings.

Active, real-time enforcement

Each match is blocked before execution — not logged after the fact.

Consistent rejection message

Every blocked attempt returns Command '<cmd>' is forbidden to the session.

```
secops@RH-LinuxAdmin: ~ (via JumpServer)

Six high-risk commands attempted — all rejected live:

$ sudo su
Command 'sudo su' is forbidden

$ rm -rf /var/lib
Command 'rm -rf /var/lib' is forbidden

$ dd if=/dev/zero of=/dev/sda
Command 'dd if=/dev/zero of=/dev/sda' is forbidden

$ mkfs.ext4 /dev/sda1
Command 'mkfs.ext4 /dev/sda1' is forbidden

$ shutdown -h now
Command 'shutdown -h now' is forbidden
```

6 / 6 BLOCKED — 0% BYPASS

Capturing the session as forensic evidence

root@SecOnion:/tmp — so-import-pcap

```
root@SecOnion:/tmp# so-import-pcap pam-lab-incident.pcap
Please wait while:

- checking /tmp/pam-lab-incident.pcap
- attempting to recover /tmp/pam-lab-incident.pcap to /tmp/so-import-pcap-IIG8TDsT7C.pcap.
- analyzing traffic with Snort
- analyzing traffic with Zeek
- processing pcap dates 2026-08-29 through 2026-08-29
  - processing 2026-08-29
    - writing /nsm/sensor_data/seconion-import/dailylogs/2026-08-29/snort.log.1787961600

- removing temporary pcap /tmp/so-import-pcap-IIG8TDsT7C.pcap

Import complete!

You can use the following hyperlink to view data in the time range of your import.  You can triple-click to quickly highlight the entire hyperli
nk and you can then copy it into your browser:
https://localhost/app/kibana#/dashboard/94b52620-342a-11e7-9d52-4f090484f59e?_g=(refreshInterval:(display:Off,pause:!f,value:0),time:(from:'2026
-08-29T00:00:00.000Z',mode:absolute,to:'2026-08-30T00:00:00.000Z'))

or you can manually set your Time Range to be:
From: 2026-08-29    To: 2026-08-30

Please note that it may take 30 seconds or more for events to appear in Kibana.
root@SecOnion:/tmp# tcpdump -i eth0 -w /tmp/metasploitable-incident.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C5440 packets captured
5440 packets received by filter
0 packets dropped by kernel
root@SecOnion:/tmp# ls
awk.conf                local_rules.xml             systemd-private-a52c581f5baa49b8bc723671bda3b0f5-colord.service-SB9WiU
config-err-jCN08v       metasploitable-incident.pcap  systemd-private-a52c581f5baa49b8bc723671bda3b0f5-rtkit-daemon.service-lzsPz9
emerging.rules.tar.gz   pam-lab-incident.pcap
emerging.rules.tar.gz.md5  ssh-ryvMUuDbOyll
```

Evidence chain

`tcpdump -i eth0 -w pam-lab-incident.pcap`

Full-content capture running for the entire enforcement test.

`so-import-pcap pam-lab-incident.pcap`

Ingested straight into Security Onion , Zeek and Snort reprocess it exactly as if it were live traffic.

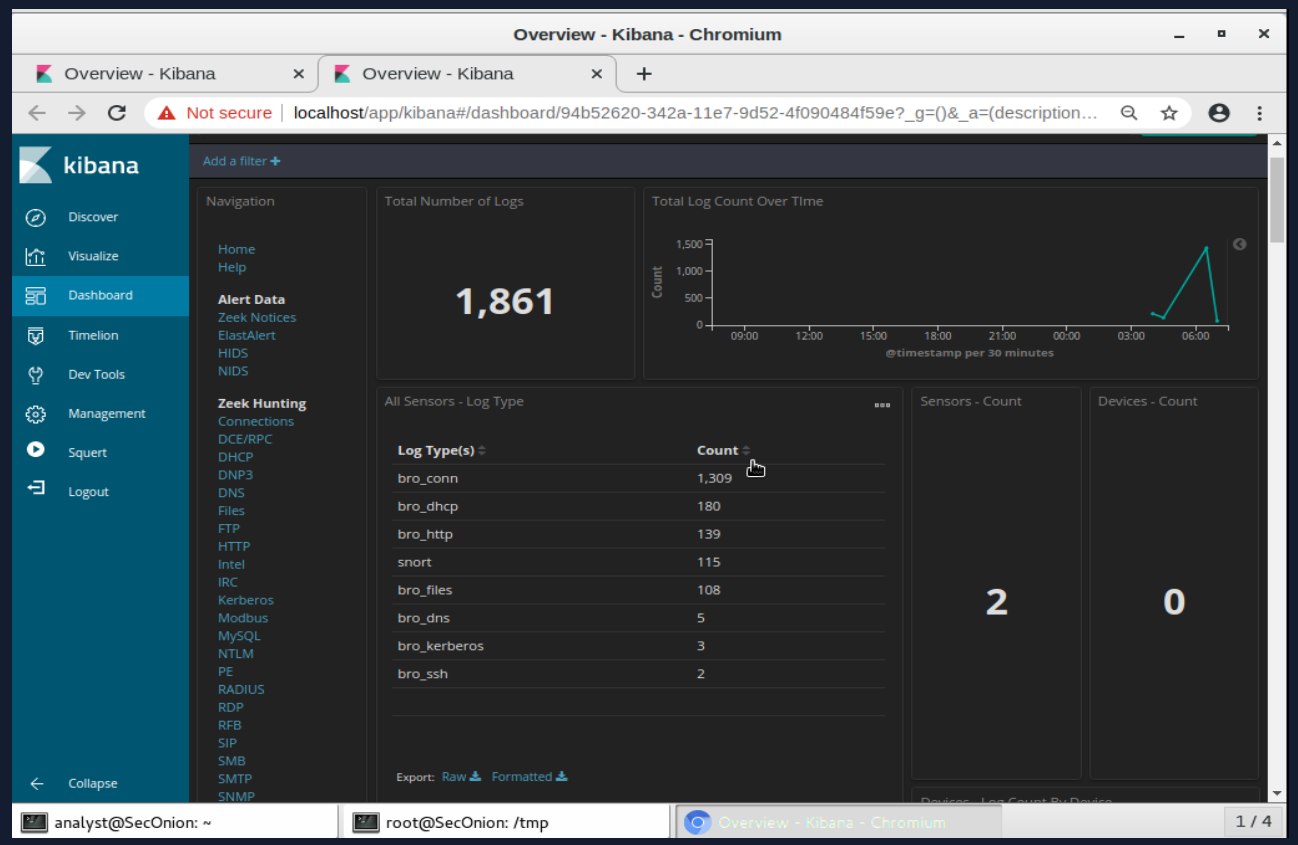
`Auto-generated Kibana link`

Import prints a direct, time bounded dashboard URL for the analyst to jump into.

Analyst note: the capture was started before the enforcement test, not after, so the evidence trail covers the full session, matching real incident-response capture discipline rather than a reconstruction after the fact.

Security Onion: what the sensor actually saw

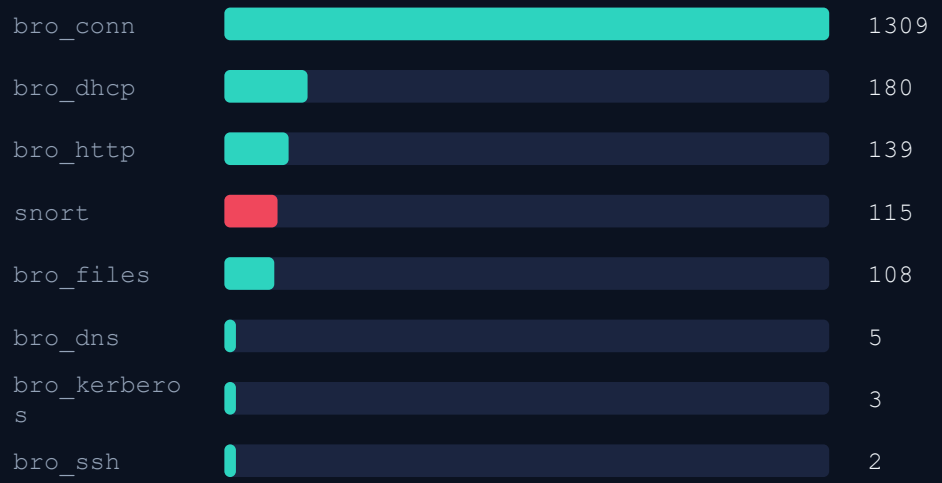
Kibana — Overview Dashboard



Sensor totals across both scenarios

1,861	115	2
total logs	Snort NIDS alerts	active sensors

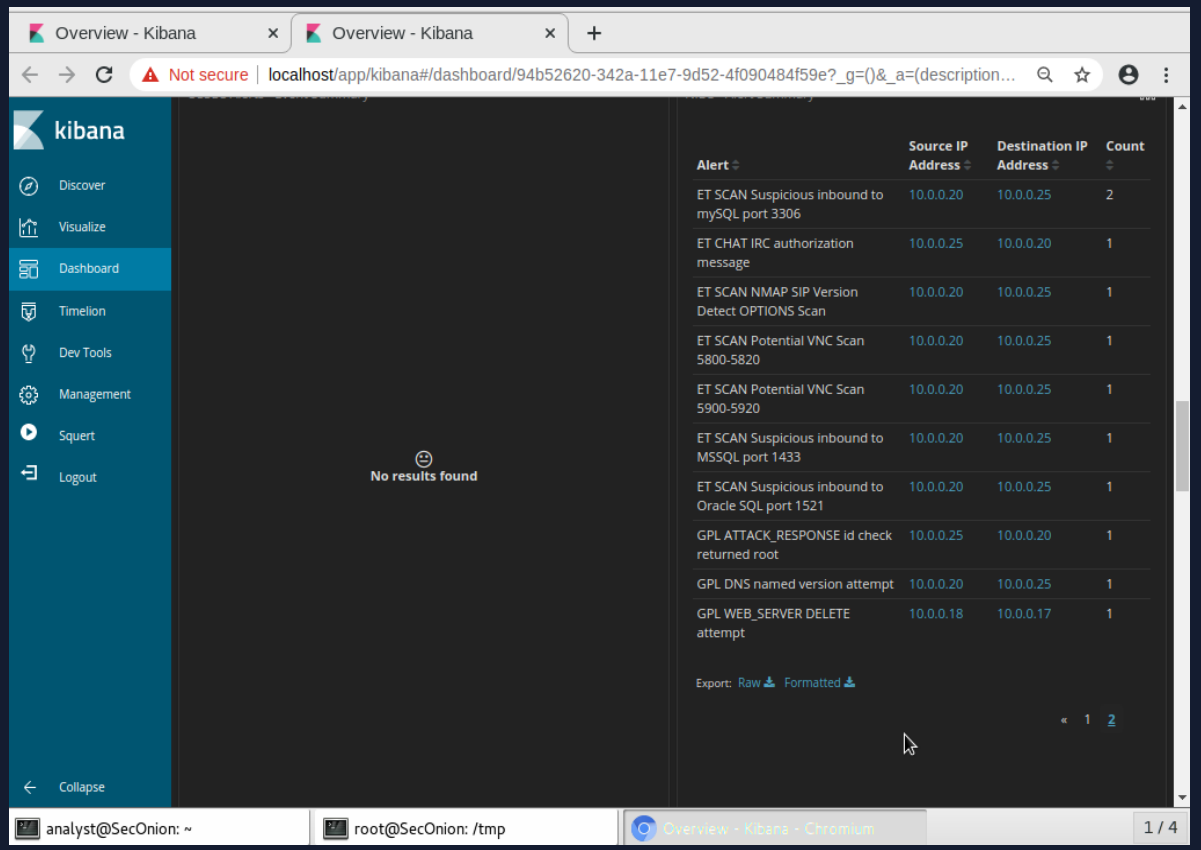
Log volume by Zeek/Snort type



bro_ssh shows just 2 connection records — metadata only. Command content inside an encrypted SSH session is invisible at the network layer (see Part 1 findings).

NIDS alert summary: 18 signatures, 115 events

Kibana — NIDS Alert Summary (2/2)



Reading the alert list like an analyst

Every distinct Snort signature is listed with its source, destination, and hit count. Before opening a single payload, the source/destination pairing alone sorts most of these into a story:

10.0.0.20 → 10.0.0.25

Kali → Metasploitable2

Recon & exploitation traffic — expected, investigate

10.0.0.25 → 10.0.0.20

Metasploitable2 → Kali

Response traffic — highest-priority alert lives here

10.0.0.17 ⇄ 10.0.0.18

JumpServer ↔ secOps

PAM application traffic — likely benign, verify

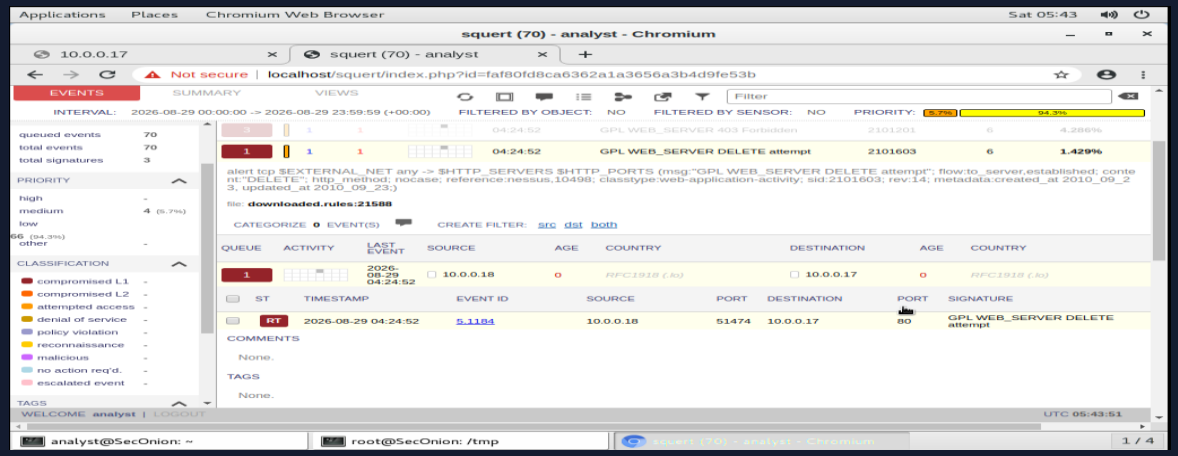
Two alerts were pulled for full payload review — one from each side of this list — shown on the next slides.

PAYLOAD REVIEW · CASE A

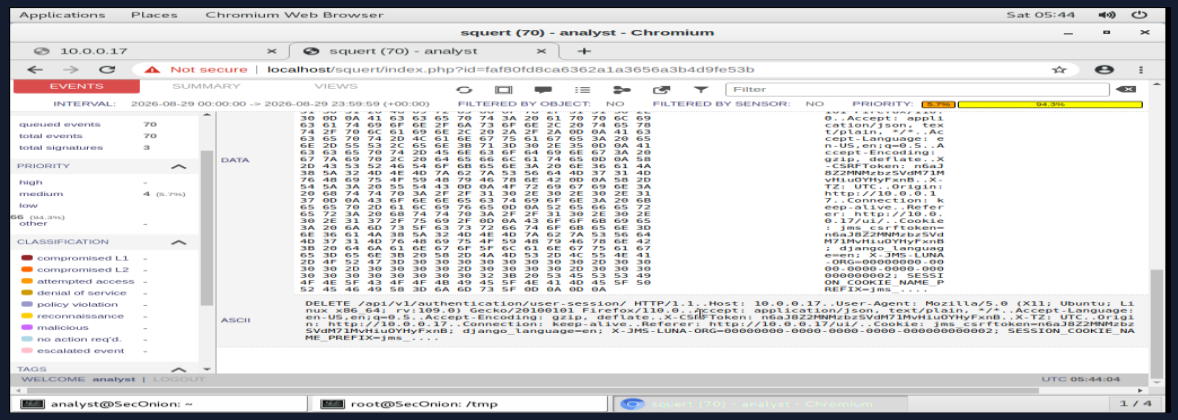
VERDICT: FALSE POSITIVE

GPL WEB_SERVER DELETE attempt

Squert — Event Detail



Squert — Raw Payload (ASCII)



What triggered it

`10.0.0.18 → 10.0.0.17 · port 80 · sid 2101603`

- Generic Snort signature keys on the literal HTTP method DELETE anywhere in a web request.
- The payload shows DELETE /api/v1/authentication-session/ — a legitimate JumpServer REST call.
- Valid session cookies, a CSRF token, and Django-style headers are all present — consistent with normal app behavior, not an attacker-crafted request.
- The matching 403 Forbidden alerts (x3) are the same story: the API correctly rejecting unauthorized calls, which Snort's generic rule also flags.

Analyst conclusion

A generic, application-agnostic web-server signature matched benign PAM management traffic. This is a textbook false positive — confirmed only by reading the raw payload, not by the alert name alone.

Building and capturing the kill chain

root@SecOnion:/tmp — capture & import

```
root@SecOnion:/tmp# tcpdump -i eth0 -w /tmp/metasploitable-incident.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
^C5440 packets captured
5440 packets received by filter
0 packets dropped by kernel
root@SecOnion:/tmp# ls
awk.conf                  local_rules.xml              systemd-private-a52c581f5baa49b8bc723671bda3b0f5-colord.service-SB9WiU
config-err-jCN08v         metasploitable-incident.pcap  systemd-private-a52c581f5baa49b8bc723671bda3b0f5-rtkit-daemon.service-lzsPz9
emerging.rules.tar.gz     pam-lab-incident.pcap
emerging.rules.tar.gz.md5  ssh-ryvMUuDbOyll
root@SecOnion:/tmp# so-import-pcap metasploitable-incident.pcap
Please wait while:

- checking /tmp/metasploitable-incident.pcap
- analyzing traffic with Snort
- analyzing traffic with Zeek
- processing pcap dates 2026-08-29 through 2026-08-29
  - processing 2026-08-29
    - copying traffic to /tmp/so-import-pcap-rhqicIgYi3.pcap
    - copying /nsm/sensor_data/seconion-import/dailylogs/2026-08-29/snort.log.1787961600 to /tmp/so-import-pcap-VIrSQkuVhy.pcap
    - merging /tmp/so-import-pcap-VIrSQkuVhy.pcap and /tmp/so-import-pcap-rhqicIgYi3.pcap into /nsm/sensor_data/seconion-import/dailylogs/2026-0
8-29/snort.log.1787961600
    - removing /tmp/so-import-pcap-VIrSQkuVhy.pcap and /tmp/so-import-pcap-rhqicIgYi3.pcap


Import complete!

You can use the following hyperlink to view data in the time range of your import.  You can triple-click to quickly highlight the entire hyperli
nk and you can then copy it into your browser:
https://localhost/app/kibana#/dashboard/94b52620-342a-11e7-9d52-4f090484f59e?_g=(refreshInterval:(display:Off,pause:!f,value:0),time:(from:'2026
-08-29T00:00:00.000Z',mode:absolute,to:'2026-08-30T00:00:00.000Z'))

or you can manually set your Time Range to be:
From: 2026-08-29    To: 2026-08-30

Please note that it may take 30 seconds or more for events to appear in Kibana.
root@SecOnion:/tmp#
```

analyst@SecOnion: ~ | root@SecOnion: /tmp | Overview - Kibana - Chromium | 1 / 4

Kill-chain stages captured

Static IP + connectivity check

Metasploitable2 assigned 10.0.0.25; bidirectional ping confirmed with Kali before any testing began.

Capture-before-attack

tcpdump started on Security Onion first, so reconnaissance is part of the evidence — not just the exploit.

Reconnaissance

Nmap service/version scan (-sV -sC) from Kali generated 10+ distinct scan-signature alerts.

Exploitation

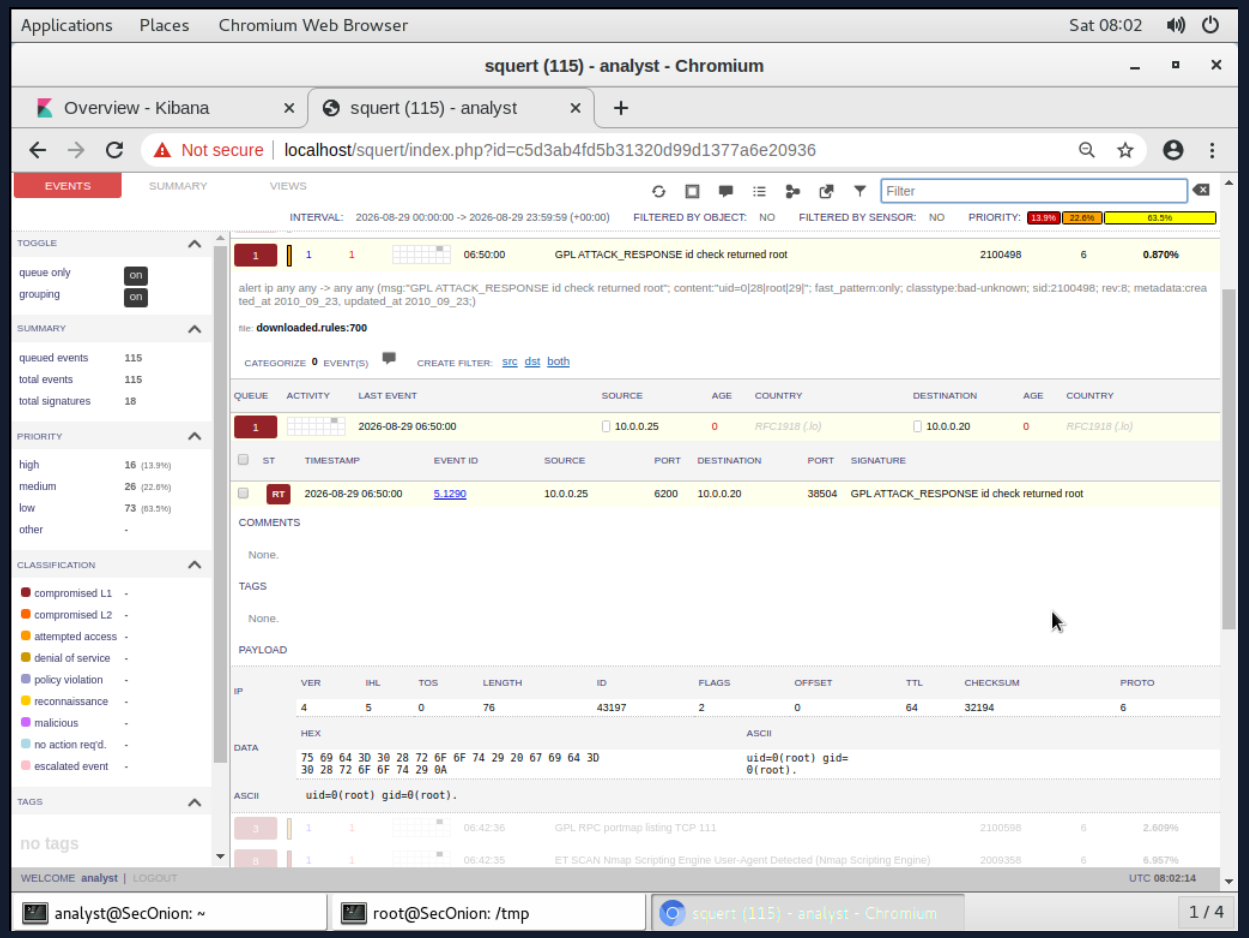
The vsftpd 2.3.4 backdoor listener (port 6200) was exploited for command execution.

PAYLOAD REVIEW · CASE B

VERDICT: TRUE POSITIVE

GPL ATTACK_RESPONSE id check returned root

Squert — Event & Raw Payload



What the payload proves

10.0.0.25 → 10.0.0.20 · port 6200 → 38504 · sid 2100498

`uid=0(root) gid=0(root).`

- This is not a scan signature — Snort matched the literal output of the id command inside response traffic returning from the victim to the attacker.
- The response originates from Metasploitable2 (.25) on port 6200 — the well-known vsftpd 2.3.4 backdoor listener.
- uid=0 / gid=0 means the command executed as root — full compromise, not partial access.
- This alert directly corroborates the manual exploitation performed from Kali, closing the loop between offensive action and network-level detection.

Analyst conclusion

Confirmed root-level compromise of Metasploitable2 via the vsftpd 2.3.4 backdoor. This is the incident's smoking-gun alert.

Verdict, side by side

ATTRIBUTE	CASE A — FALSE POSITIVE	CASE B — TRUE POSITIVE
Signature	GPL WEB_SERVER DELETE attempt	GPL ATTACK_RESPONSE id check returned root
Traffic path	10.0.0.18 → 10.0.0.17 (secOps → JumpServer)	10.0.0.25 → 10.0.0.20 (Metasploitable2 → Kali)
Root cause	Legitimate REST call matched a generic rule	vsftpd 2.3.4 backdoor exploited for root RCE
Payload evidence	Valid session cookie, CSRF token, JSON body	Literal command output: "uid=0(root) gid=0(root)."
Analyst action	Document as benign; tune/allowlist the signature	Escalate as confirmed compromise; begin IR

The same triage process — alert → payload → context → verdict — applies to both cases. The signature name alone was never enough to decide either verdict.

SKILLS DEMONSTRATED

What this project put into practice

PAM hardening

Deployed and configured a bastion host with live command-filtering policy enforcement.

Network security monitoring

Operated Zeek + Snort + Squert + Kibana on Security Onion to triage real alerts.

Offensive fundamentals

Executed a minimal, controlled recon-to-exploitation chain against a known-vulnerable host.

Payload-level analysis

Read raw hex/ASCII payloads to move from alert name to defensible verdict.

True/false positive triage

Applied source/destination context and evidence, not signature names, to classify alerts.

Evidence discipline

Captured before attacking, preserved full pcaps, and documented the chain of analysis.

Thank you